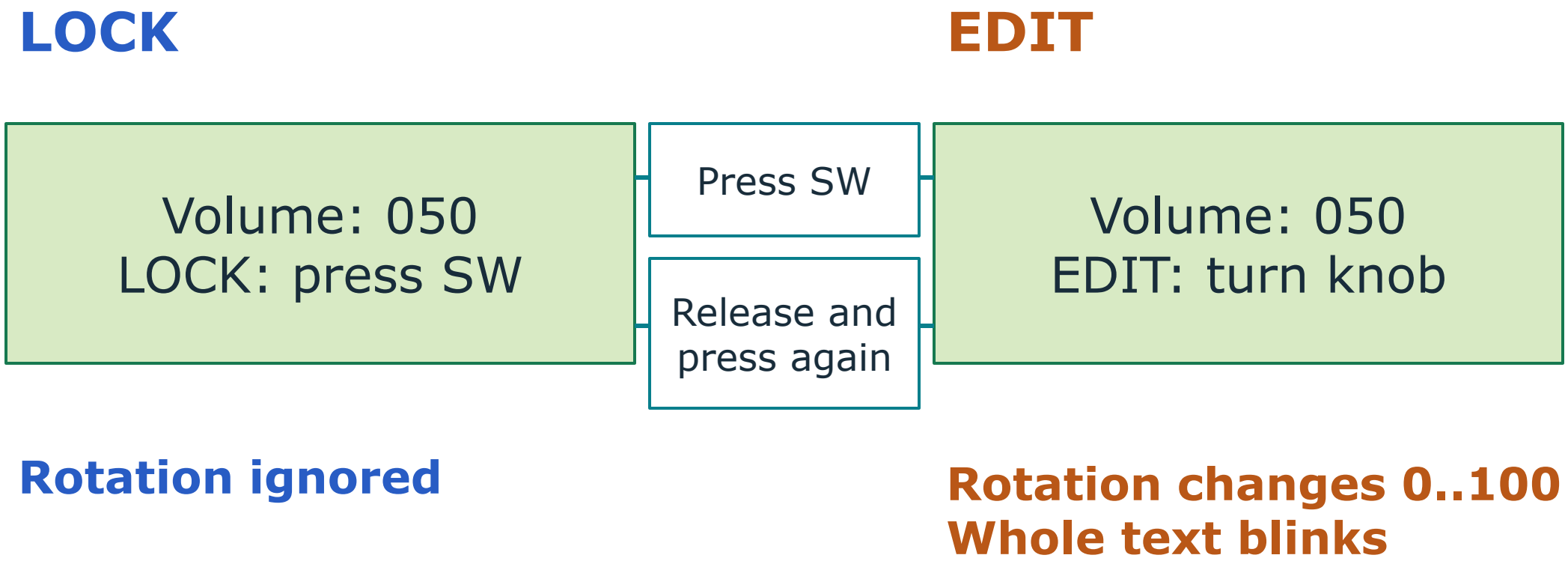
AVR external interrupts

Session 7
Rotary encoder volume control
ATmega328P on Arduino Uno

The control we will build



Learning route

Input signals and interrupt execution

INT0/INT1, short ISRs and shared variables

Atomic access, debounce and encoder direction

PCINT groups, LCD control, then one complete demonstration

Final assignment: the same behavior using PCINT only

Pin names and bit positions

Name	Meaning	Example
D2	Uno board label	Connected to PD2 / INT0
PD2	Bit position 2 in port D	1 << PD2 is 00000100
PIND	Input register for port D	Read the physical pin levels
PORTD	Output latch / input pull-up control	Set a bit to enable its pull-up
DDRD	Direction register	0 input, 1 output

Input and pull-up configuration

```
#include <avr/io.h>

DDRD &= ~(1 << PD2);
PORTD |= (1 << PD2);

uint8_t now = (PIND >> PD2) & 1;
```

PD2 = 2 comes from <avr/io.h>. Include <stdint.h> for uint8_t.

A button to GND reads 1 when released and 0 when pressed.

Polling with software edge detection

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

uint8_t previous = 0, now = 0, count = 0;
while (1) {
    now = (PIND >> PD2) & 1;
    if (previous == 1 && now == 0) count++;
    previous = now;
    _delay_ms(250);
}
```

Loop fragment. All three variables start at 0. PD2 = 2 comes from <avr/io.h>.

F_CPU describes 16 MHz for delay calculations. UL means unsigned long.

One sampled press and release

Iteration	Button	Previous	Now	Count
1	Released	0	1	0
2	Released	1	1	0
3	Pressed	1	0	1
4	Held	0	0	1
5	Released	0	1	1

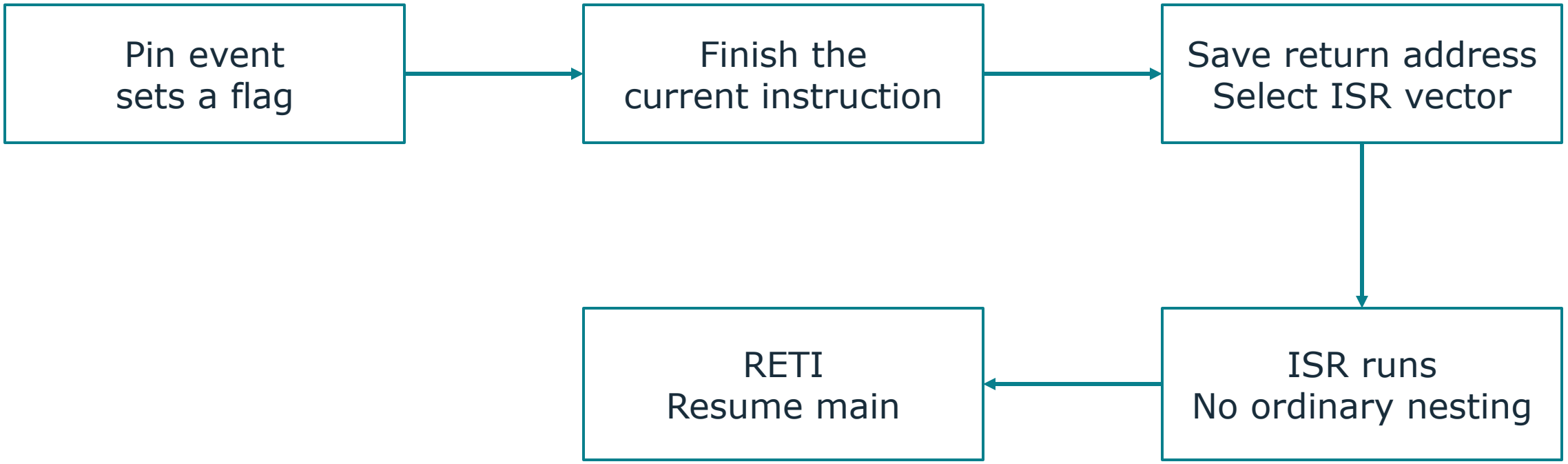
Polling and interrupt capture



Polling reads HIGH at both samples.

An enabled falling-edge interrupt captures the event in between.

Interrupt execution flow



Masks and SREG.I must permit service.

The interrupt enable conditions

The pin hardware recognizes the configured event.

The source has an enabled mask bit.

SREG.I enables global interrupts after sei().

The CPU runs the ISR, then resumes the interrupted program.

Repeated events can merge into one pending flag.

Dedicated external interrupt inputs

Uno pin	AVR pin	Source	ISR name
D2	PD2	INT0	INT0_vect
D3	PD3	INT1	INT1_vect

EICRA trigger selection

ISCn1	ISCn0	Trigger
0	0	LOW level
0	1	Any logical change
1	0	Falling edge
1	1	Rising edge

A falling-edge INT0 configuration

```
#include <avr/io.h>
#include <avr/interrupt.h>

cli();
EICRA = (1 << ISC01);
EIFR = (1 << INTF0);
EIMSK = (1 << INT0);
sei();
```

Header constants: ISC01 = 1, INTF0 = 0, INT0 = 0.

EIFR clears a flag by writing 1. Use assignment for this clear operation.

Any change on INT0 and INT1

```
#include <avr/io.h>
#include <avr/interrupt.h>

EICRA = (1 << ISC00) | (1 << ISC10);
EIFR = (1 << INTF0) | (1 << INTF1);
EIMSK = (1 << INT0) | (1 << INT1);
```

ISC00 = 0, ISC10 = 2. EICRA becomes 00000101.

INTF0 / INT0 = 0. INTF1 / INT1 = 1. Enable global interrupts last.

A short interrupt service routine

```
#include <avr/interrupt.h>
#include <stdint.h>

volatile uint8_t activity = 0;

ISR(INT0_vect)
{
   activity = 1;
}
```

ISR and INT0_vect come from <avr/interrupt.h>.

The handler records activity. Main performs slower processing.

volatile and shared state

An ISR can change a variable between statements in main.

volatile tells the compiler to perform the required accesses.

It does not make several statements indivisible.

Each shared operation still needs an atomicity check.

An event can disappear between two statements

```
volatile uint8_t switch_activity = 0;
uint8_t activity;

activity = switch_activity;
// An ISR may set switch_activity here.
switch_activity = 0;
```

Both accesses are 8-bit, but the combined operation can lose a new event.

The next diagram shows the exact order.

A concrete lost-event sequence

main	**activity = switch_activity**	Copies 0
ISR	**switch_activity = 1**	A new edge arrives
main	**switch_activity = 0**	New activity is erased

The copy and clear must happen together.

Atomic copy and clear

```
#include <util/atomic.h>

ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
    activity = switch_activity;
    switch_activity = 0;
}
```

Both macros come from <util/atomic.h>. RESTORESTATE restores the earlier I bit.

Keep the block short. LCD writes and delays stay outside it.

A 16-bit read can mix two values

Moment	Counter / action	Result
Before interrupt	Counter = 0x00FF	255
Main first read	Reads low byte 0xFF	Low byte saved
ISR	Increments counter to 0x0100	256
Main second read	Reads high byte 0x01	Combined 0x01FF = 511

Mode changes also form one transaction

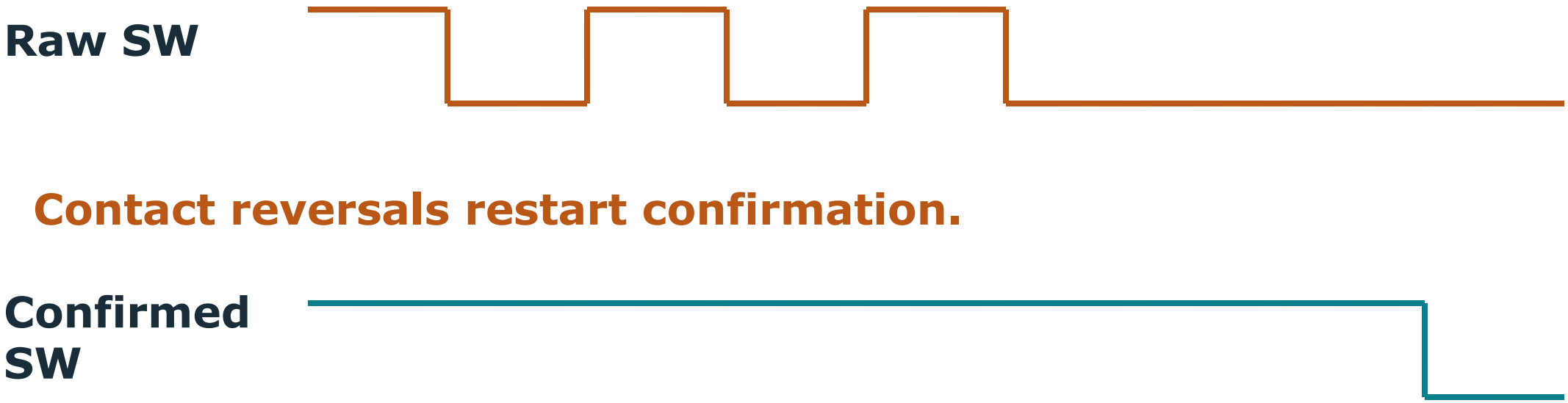
```
#include <util/atomic.h>

ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
    editing = !editing;
    encoder_quarters = 0;
    encoder_previous = encoder_read();
}
```

The ISR must see a consistent mode and decoder starting state.

encoder_read() samples CLK and DT. Its complete definition appears later.

Mechanical button bounce



Incorrect behavior without debounce

Input event	Raw falling-edge count	Wanted press count
First contact closure	1	1
Contact rebounds then closes	2	1
Another rebound and closure	3	1
Button remains held	3	1

Simple debounce variables

```
#include <stdint.h>
#define DB_SAMPLES 20

uint8_t switch_stable = 1;
uint8_t switch_candidate = 1;
uint8_t switch_samples = 0;
uint8_t switch_tracking = 1;
```

stable is confirmed. candidate is being checked. samples counts matching intervals.

At about 1 ms per call, 20 matching intervals take about 20 ms.

Debounce starts after activity

```
if (activity != 0) {
    switch_tracking = 1;
}
if (switch_tracking == 0) {
    return 0;
}

uint8_t raw_high = (PINB >> PB0) & 1;
```

PB0 = 0 comes from <avr/io.h>. activity comes from atomic copy-and-clear.

return 0 exits this function with no newly confirmed press.

A changed sample restarts the check

```
#define DB_SAMPLES 20

if (raw_high != switch_candidate) {
    switch_candidate = raw_high;
    switch_samples = 0;
} else if (switch_samples < DB_SAMPLES) {
    switch_samples++;
}
```

A new candidate starts at zero matching intervals.

A bounce changes the candidate and restarts the count.

A confirmed press produces one event

```
#define DB_SAMPLES 20

if (switch_samples < DB_SAMPLES) return 0;
switch_tracking = 0;
if (switch_candidate == switch_stable) return 0;
switch_stable = switch_candidate;
if (switch_stable == 0) return 1;
return 0;
```

LOW reports a new press. HIGH confirms release without reporting a press.

Holding LOW keeps the confirmed state LOW and produces no repeated event.

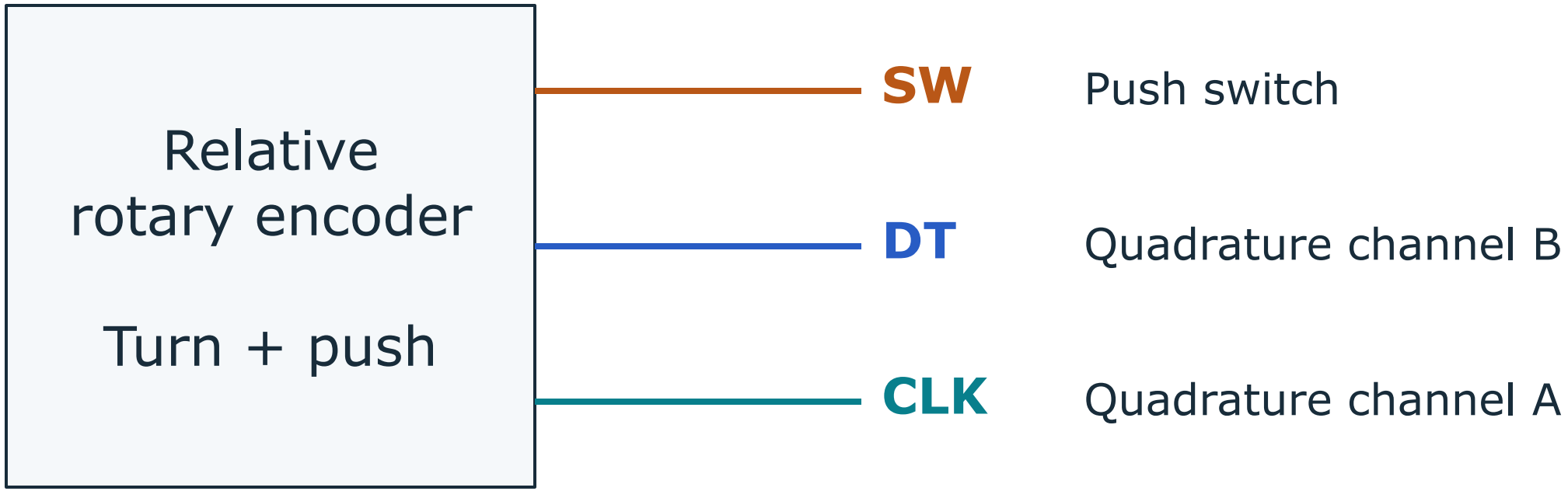
Numerical debounce trace

Time	SW reading	Candidate	Matching intervals	Action
0 ms	1	1	Already accepted	Released
1 ms	0	0	0	Restart
2 ms	1	1	0	Bounce: restart
3 ms	0	0	0	Restart
4..22 ms	0	0	1..19	Wait
23 ms	0	0	20	Accept one press

Hold and release behavior

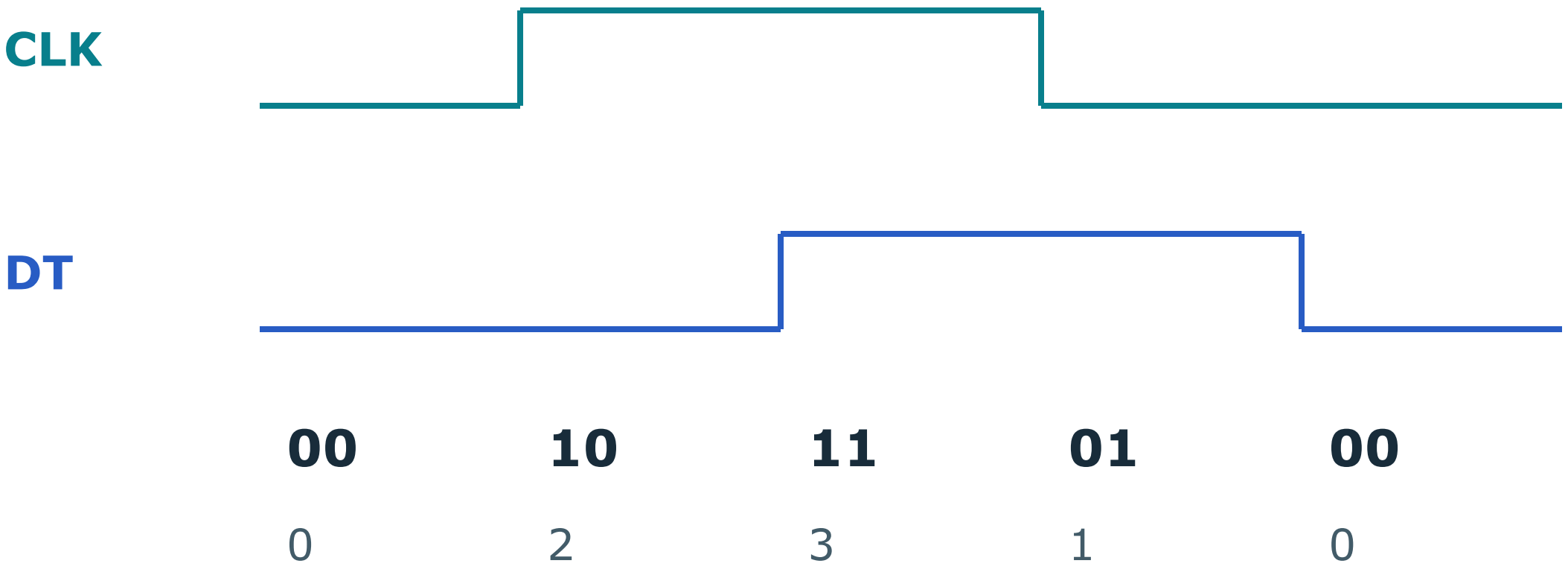
Situation	Confirmed state	Press result
Press stable for ~20 ms	LOW	1 once
Hold for five seconds	LOW	0
Release stable for ~20 ms	HIGH	0
Press again and confirm	LOW	1 once

The rotary encoder signals



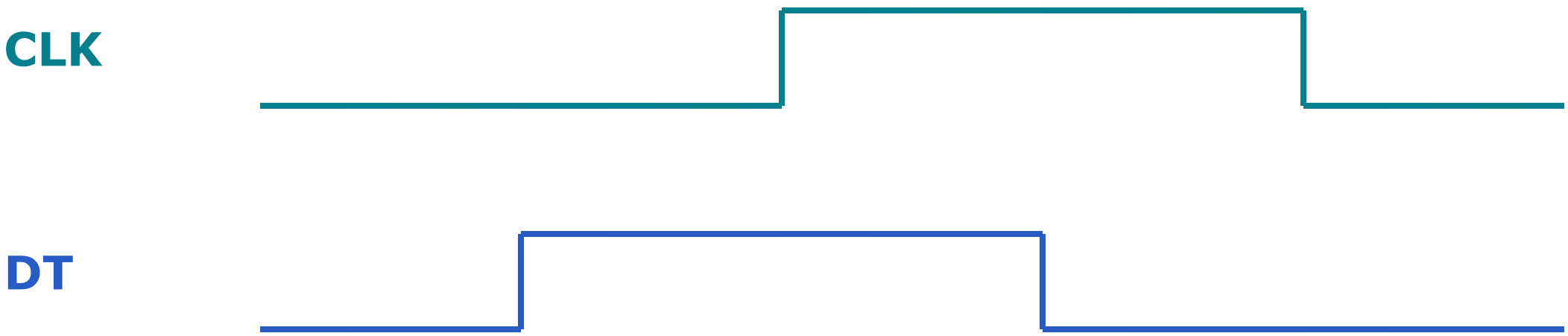
The SimulIDE model exposes three logic outputs. Physical modules also need power.

Clockwise quadrature sequence



State = (CLK << 1) | DT. One valid transition changes one bit.

Counterclockwise quadrature sequence



00	01	11	10	00
0	1	3	2	0

State = (CLK << 1) | DT. One valid transition changes one bit.

Reading both encoder signals

```
#include <avr/io.h>
#include <stdint.h>

uint8_t pins = PIND;
uint8_t clk = (pins >> PD2) & 1;
uint8_t dt = (pins >> PD3) & 1;
uint8_t state = (clk << 1) | dt;
```

PD2 = 2 and PD3 = 3 are header bit positions.

One PIND read captures both levels before extracting the two bits.

Four transitions form one volume step

Transition	Quarter count	Volume from 50
0 to 2	1	50
2 to 3	2	50
3 to 1	3	50
1 to 0	4, then reset to 0	51

Direction decoding without a lookup table

```
#define DIRECTION_SIGN 1

if ((previous == 3 && now == 1) ||
    (previous == 1 && now == 0) ||
    (previous == 0 && now == 2) ||
    (previous == 2 && now == 3)) {
    encoder_quarters += DIRECTION_SIGN;
}
```

These four neighboring transitions are the clockwise cycle.

The opposite four transitions subtract. An invalid two-bit jump resets progress.

A full cycle changes volume once

```
if (encoder_quarters == 4) {
    if (volume < 100) volume++;
    encoder_quarters = 0;
} else if (encoder_quarters == -4) {
    if (volume > 0) volume--;
    encoder_quarters = 0;
}
```

The range is 0 to 100. A reverse step works immediately at either limit.

When editing is 0, the decoder clears partial progress and changes no volume.

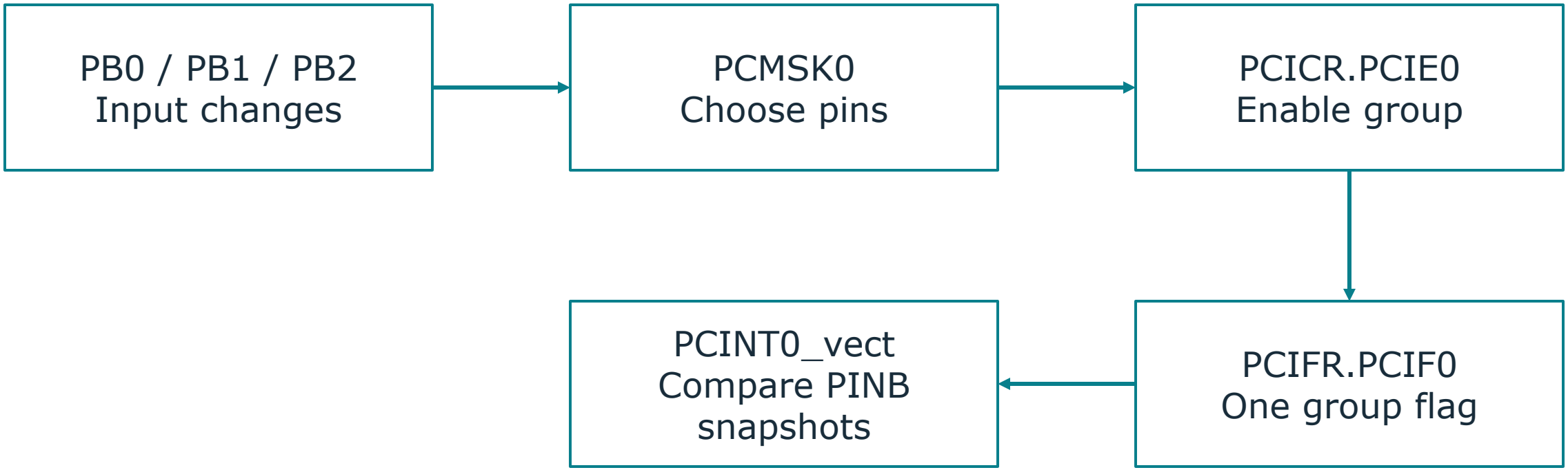
Encoder filtering and switch debounce

Signal	Method	Concrete example
SW	Confirm one level for ~20 ms	Reject LOW for only 5 ms
CLK and DT	Validate quadrature transitions	0 to 2 to 0 cancels to zero
CLK and DT	Reject impossible sampled jumps	0 to 3 resets progress

ATmega328P PCINT groups

Port	PCINT sources	Enable / mask	Shared ISR
B	PCINT0..7	PCIE0 / PCMSK0	PCINT0_vect
C	PCINT8..14	PCIE1 / PCMSK1	PCINT1_vect
D	PCINT16..23	PCIE2 / PCMSK2	PCINT2_vect

PCINT group and pin selection



The group flag does not name the changed pin.

SW on D8 uses PCINT0

```
#include <avr/io.h>
#include <avr/interrupt.h>

DDRB &= ~(1 << PB0);
PORTB |= (1 << PB0);
previous_port_b = PINB;
PCMSK0 = (1 << PCINT0);
PCIFR = (1 << PCIF0);
PCICR = (1 << PCIE0);
```

PB0, PCINT0, PCIF0 and PCIE0 all have bit-position value 0.

Set the baseline before enabling the group. Global enable comes last.

XOR identifies changed input bits

Value	PB2 PB1 PB0	Meaning
Previous	0 0 1	Only SW HIGH
Current	1 1 0	DT and CLK HIGH, SW LOW
XOR	1 1 1	All three bits differ
Mask with 0x07	1 1 1	Keep the three selected inputs

Shared-vector source identification

```
#include <avr/io.h>

uint8_t now = PINB;
uint8_t changed = (now ^ previous_port_b) & 0x07;
previous_port_b = now;
if ((changed & (1 << PB0)) != 0) {
    switch_activity = 1;
}
if ((changed & ((1 << PB1) | (1 << PB2))) != 0) {
    // Decode CLK and DT from this same now snapshot.
}
```

PB0 = 0, PB1 = 1, PB2 = 2. Mask 0x07 is binary 00000111.

Independent if statements handle a switch change and rotation together.

The LCD connection uses 4-bit transfers

LCD signal	Uno pin	AVR pin
RS	D12	PB4
E / En	D11	PB3
DB4, DB5	D4, D5	PD4, PD5
DB6, DB7	D6, D7	PD6, PD7
R/W	GND	Write only
DB0..DB3	Unconnected	Unused in 4-bit mode

Named LCD pins have visible definitions

```
#include <avr/io.h>
#define LCD_RS PB4
#define LCD_EN PB3

PORTB &= ~(1 << LCD_RS);
PORTB |= (1 << LCD_EN);
// E goes LOW after the required enable pulse.
PORTB &= ~(1 << LCD_EN);
```

PB4 = 4 and PB3 = 3 come from <avr/io.h>.

LCD_RS and LCD_EN are aliases defined above, not hidden pin numbers.

LCD writes preserve the input pull-ups

```
PORTD = (PORTD & 0x0F)
      | ((value & 0x0F) << 4);
```

0x0F keeps PD0..PD3. The outgoing nibble occupies PD4..PD7.

Writing PORTD = value would damage unrelated pin settings.

Foreground and interrupt responsibilities

Interrupt handlers

Sample CLK and DT
Validate one transition
Update bounded volume
Record SW activity

Short, immediate work

Foreground loop

Atomic copy and clear
Confirm SW press / release
Toggle mode consistently
Refresh and blink the LCD

Slower display and timing work

Demonstration behavior

Startup: Volume 050, LOCK mode, text steady.

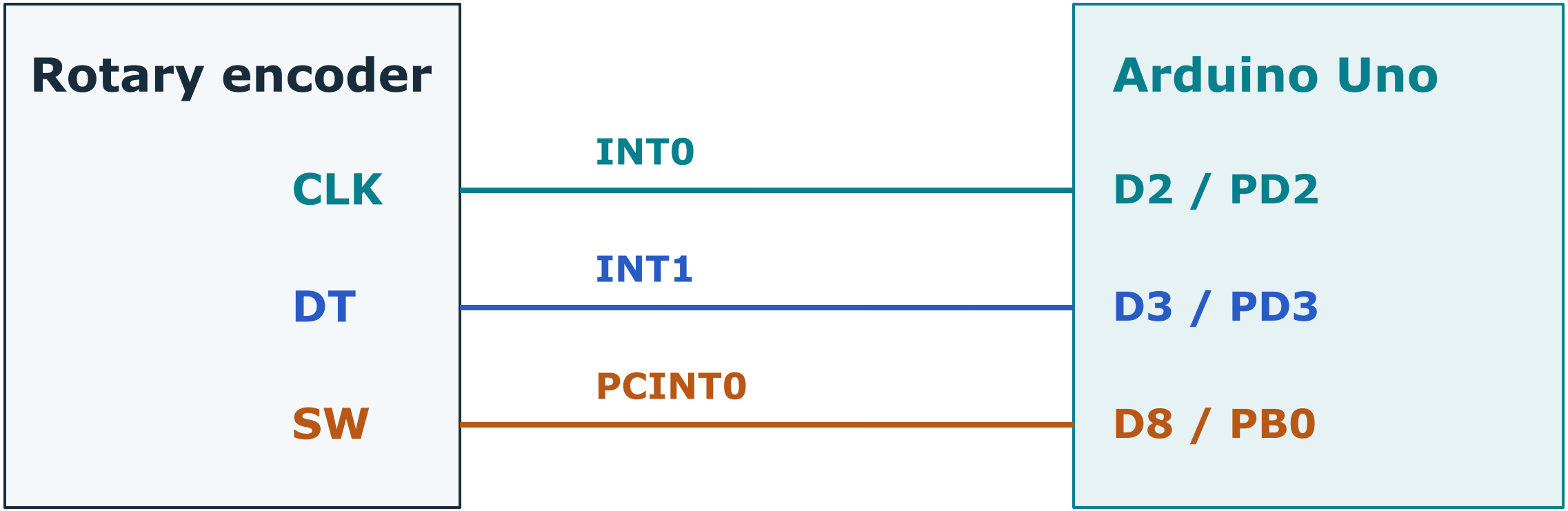
Press and release SW: EDIT mode, text blinks.

Turn clockwise to increase and counterclockwise to decrease.

Press and release SW again: LOCK mode, text steady.

Rotation while locked leaves the volume unchanged.

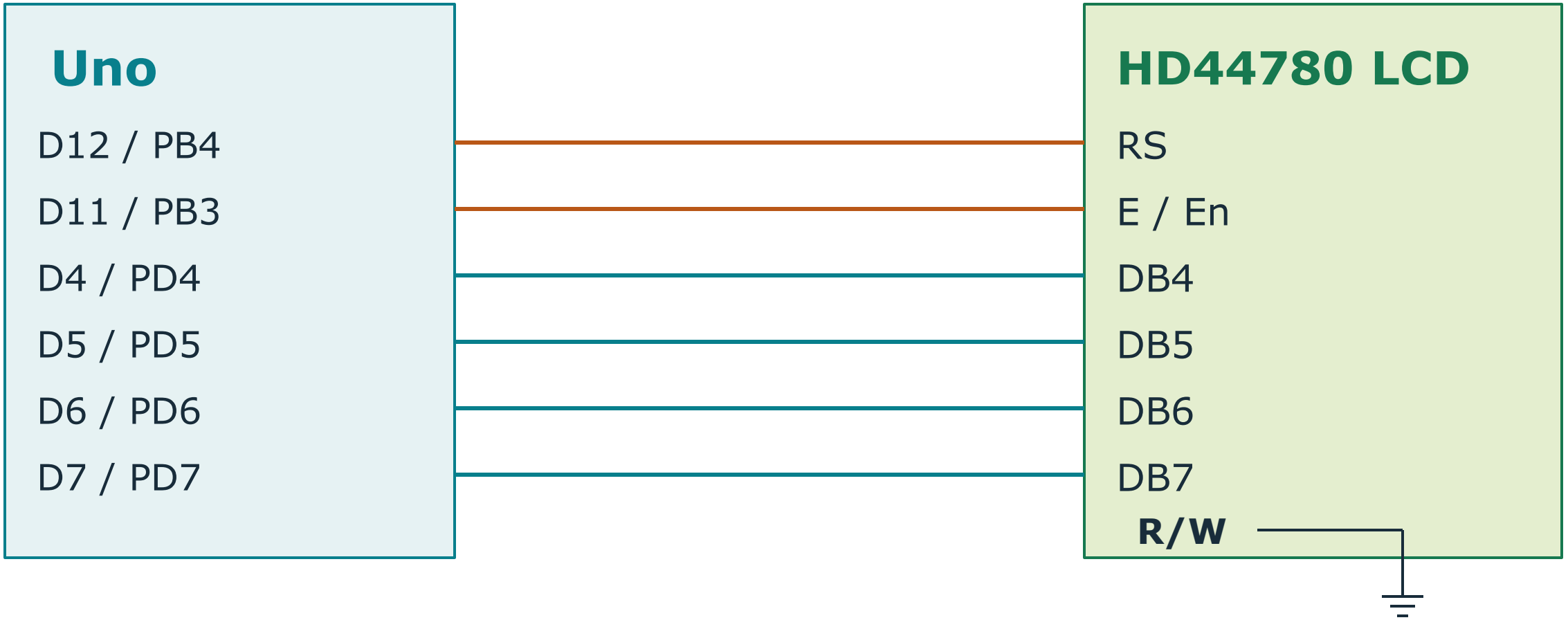
Demonstration encoder connections



CLK and DT use separate INT vectors. SW uses PCINT0_vect.

Internal pull-ups ON. The SimulIDE encoder has no exposed power pins.

LCD connections for both projects



DB0..DB3 remain unconnected. Set 16 columns and 2 rows.

Demonstration interrupt configuration

```
#include <avr/io.h>

EICRA = (1 << ISC00) | (1 << ISC10);
EIFR = (1 << INTF0) | (1 << INTF1);
PCMSK0 = (1 << PCINT0);
PCIFR = (1 << PCIF0);
PCICR = (1 << PCIE0);
EIMSK = (1 << INT0) | (1 << INT1);
```

ISC00=0, ISC10=2. INTF0/INT0=0, INTF1/INT1=1.

PCINT0, PCIF0, PCIE0 = 0. Configure baselines before these lines.

Demonstration interrupt handlers

```
#include <avr/interrupt.h>

ISR(INT0_vect) { encoder_update(encoder_read()); }
ISR(INT1_vect) { encoder_update(encoder_read()); }

ISR(PCINT0_vect)
{
   uint8_t now = PINB;
   uint8_t changed = (now ^ previous_port_b) & 1;
   previous_port_b = now;
   if (changed != 0) switch_activity = 1;
}
```

ISR and vector names come from <avr/interrupt.h>. Mask 1 selects PB0.

The complete source uses (1 << PB0), with PB0 = 0.

PlatformIO project configuration

```
[env]
platform = atmelavr
board = uno
board_build.f_cpu = 16000000L
build_flags = -std=gnu11 -Wall -Wextra
```

Open Firmware as the project. Run pio run -e demo.

Load .pio/build/demo/firmware.hex into the Uno. Set its clock to 16 MHz.

Student assignment

Build the same volume controller using PCINT for every encoder input.

Keep the same LOCK/EDIT behavior, blink, debounce and 0..100 limits.

Choose three PCINT pins and draw the complete connections.

Write bare-metal AVR C, test in SimulIDE, and record the results.

INT0/INT1 must remain disabled.

Assignment deliverables

A labeled encoder and LCD connection diagram

A register explanation for PCICR, PCMSKx and PCIFR

Complete commented AVR C and PlatformIO configuration

A test log covering lock, edit, both directions, hold, blink and limits

An explanation of how the shared ISR identifies changed pins

What changes between the two versions

Part	Demonstration	Assignment solution
CLK and DT pins	D2 / D3	D9 / D10
SW pin	D8	D8
Dedicated interrupts	INT0 and INT1 enabled	EIMSK = 0
PCMSK0	Bit 0	Bits 0, 1 and 2
Interrupt handlers	INT0, INT1 and PCINT0	PCINT0 only
Debounce, decoder and LCD	Same behavior	Same behavior